



Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN
BUSINESS ANALYTICSCarnegie Mellon University
Tepper School of Business

Go from Data to Strategy: Tepper School of Business

Use PyTorch Deep Learning Models with scikit-learn

by [Adrian Tam](#) on [April 8, 2023](#) in [Deep Learning with PyTorch](#) 10

Share

Share

Post

The most popular deep learning libraries in Python for research and development are TensorFlow/Keras and PyTorch, due to their simplicity. The scikit-learn library, however, is the most popular library for general machine learning in Python. In this post, you will discover how to use deep learning models from PyTorch with the scikit-learn library in Python. This will allow you to leverage the power of the scikit-learn library for tasks like model evaluation and model hyper-parameter optimization. After completing this lesson you will know:

- How to wrap a PyTorch model for use with the scikit-learn machine learning library
- How to easily evaluate PyTorch models using cross-validation in scikit-learn
- How to tune PyTorch model hyperparameters using grid search in scikit-learn

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Use PyTorch Deep Learning Models with scikit-learn
Photo by Priyanka Neve. Some rights reserved.

Overview

This chapter is in four parts; they are:

- Overview of scorch
- Evaluate Deep Learning Models with Cross-Validation
- Running k-Fold Cross-validation with scikit-learn
- Grid Search Deep Learning Model Parameters

Overview of skorch

PyTorch is a popular library for deep learning in Python, but the focus of the library is deep learning, not all of machine learning. In fact, it strives for minimalism, focusing on only what you need to quickly and simply define and build deep learning models. The scikit-learn library in Python is built upon the SciPy stack for efficient numerical computation. It is a fully featured library for general purpose machine learning and provides many useful utilities in developing deep learning models. Not least of which are:

- Evaluation of models using resampling methods like k-fold cross-validation
- Efficient search and evaluation of model hyperparameters
- Connecting multiple steps of a machine learning workflow into a pipeline

PyTorch cannot work with scikit-learn directly. But thanks to the duck-typing nature of Python language, it is easy to adapt a PyTorch model for use with scikit-learn. Indeed, the skorch module is built for this purpose. With skorch, you can make your PyTorch model work just like a scikit-learn model. You may find it easier to use.

In the following sections, you will work through examples of using the `NeuralNetClassifier` wrapper for a classification neural network created in PyTorch and used in the scikit-learn library. The test problem is the [Sonar dataset](#). This is a small dataset with all numerical attributes that is easy to work with.

The following examples assume you have successfully installed PyTorch, skorch, and scikit-learn. If you use the pip for your Python modules, you may install them with:

```
1 pip install torch skorch scikit-learn
```

Evaluate Deep Learning Models with Cross-Validation

The `NeuralNet` class, or more specialized `NeuralNetClassifier`, `NeuralNetBinaryClassifier`, and `NeuralNetRegressor` classes in skorch are factory wrappers for PyTorch models. They take an argument `model` which is a class or a function to call to get your model. In return, these wrapper classes allows you to specify loss function and optimizer, then the training loop comes for free. This is the convenience compare to using PyTorch directly.

Below is a simple example of training a binary classifier on the Sonar dataset:

```
1 import copy
2
3 import numpy as np
4 from sklearn.model_selection import StratifiedKFold, train_test_split
5
6 import pandas as pd
7 import torch
8 import torch.nn as nn
9 import torch.optim as optim
10 from sklearn.preprocessing import LabelEncoder
11 from skorch import NeuralNetBinaryClassifier
12
13 # Read data
14 data = pd.read_csv("sonar.csv", header=None)
15 X = data.iloc[:, 0:60]
16 y = data.iloc[:, 60]
17
18 # Binary encoding of labels
19 encoder = LabelEncoder()
20 encoder.fit(y)
21 y = encoder.transform(y)
22
23 # Convert to 2D PyTorch tensors
24 X = torch.tensor(X.values, dtype=torch.float32)
25 y = torch.tensor(y, dtype=torch.float32)
26
27 # Define the model
28 class SonarClassifier(nn.Module):
29     def __init__(self):
30         super().__init__()
31         self.layer1 = nn.Linear(60, 60)
32         self.act1 = nn.ReLU()
33         self.layer2 = nn.Linear(60, 60)
34         self.act2 = nn.ReLU()
35         self.layer3 = nn.Linear(60, 60)
36         self.act3 = nn.ReLU()
37         self.output = nn.Linear(60, 1)
38
39     def forward(self, x):
40         x = self.act1(self.layer1(x))
41         x = self.act2(self.layer2(x))
42         x = self.act3(self.layer3(x))
43         x = self.output(x)
44         return x
45
46 # create the skorch wrapper
47 model = NeuralNetBinaryClassifier(
48     SonarClassifier,
49     criterion=torch.nn.BCEWithLogitsLoss,
```

```

50 optimizer=torch.optim.Adam,
51 lr=0.0001,
52 max_epochs=150,
53 batch_size=10
54 )
55
56 # run
57 model.fit(X, y)

```

In this model, you used `torch.nn.BCEWithLogitsLoss` as the loss function (that is indeed the default of `NeuralNetBinaryClassifier`). It is to combine the sigmoid function with binary cross entropy loss, so that you don't need to put the sigmoid function at the output of the model. It is sometimes preferred to provide better numerical stability.

In addition, you specified the training parameters such as the number of epochs and batch size in the `skorch` wrapper. Then you just need to call `fit()` function with the input feature and target. The wrapper will help you initialize a model and train it.

Running the above will produce the following:

	epoch	train_loss	valid_acc	valid_loss	dur
1					
2					
3	1	0.6952	0.5476	0.6921	0.0135
4	2	0.6930	0.5476	0.6920	0.0114
5	3	0.6925	0.5476	0.6919	0.0104
6	4	0.6922	0.5238	0.6918	0.0118
7	5	0.6919	0.5238	0.6917	0.0112
8	...				
9	146	0.2942	0.4524	0.9425	0.0115
10	147	0.2920	0.4524	0.9465	0.0123
11	148	0.2899	0.4524	0.9495	0.0112
12	149	0.2879	0.4524	0.9544	0.0121
13	150	0.2859	0.4524	0.9583	0.0118

Note that `skorch` is positioned as a wrapper for PyTorch models to adapt to scikit-learn interface. Therefore, you should use the model as if it is a scikit-learn model. For example, to train your binary classification model, it is expected the target to be a vector rather than an $n \times 1$ matrix. And to run the model for inference, you should use `model.predict(X)` or `model.predict_proba(X)`. It is also why you should use `NeuralNetBinaryClassifier`, such that the classification-related scikit-learn functions are provided as model methods.

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Running k-Fold Cross-validation with scikit-learn

Using a wrapper over your PyTorch model already save you a lot of boilerplate code on building your own training loop. But the entire suite of machine learning functions from scikit-learn is the real productivity boost.

One example is to use the model selection functions from scikit-learn. Let's say you want to evaluate this model design with k-fold cross-validation. Normally, it means to take a dataset, split it into k portions, then run a loop to select one of these portion as test set and the rest as training set to train a model from scratch and obtain an evaluation score. It is not difficult to do but you need to write several lines of code to implement these.

Indeed, we can make use of the k-fold and cross validation function from scikit-learn, as follows:

```

1 from sklearn.model_selection import StratifiedKFold
2 from sklearn.model_selection import cross_val_score
3
4 model = NeuralNetBinaryClassifier(
5     SonarClassifier,
6     criterion=torch.nn.BCEWithLogitsLoss,
7     optimizer=torch.optim.Adam,
8     lr=0.0001,
9     max_epochs=150,
10    batch_size=10,
11    verbose=False
12 )
13
14 kfold = StratifiedKFold(n_splits=5, shuffle=True)
15 results = cross_val_score(model, X, y, cv=kfold)
16 print(results)

```

The parameter `verbose=False` in `NeuralNetBinaryClassifier` is to stop the display of progress while the model is trained, since there was a lot. The above code will print the validation score, as follows:

```
1 [0.76190476 0.76190476 0.78571429 0.75609756 0.75609756]
```

These are the evaluation scores. Because it is a binary classification model, they are the average accuracy. There are five of them because it is obtained from a k-fold cross-validation with $k = 5$, each for a different test set. Usually you evaluate a model with the mean and standard deviation of the cross-validation scores:

```
1 print("mean = %.3f; std = %.3f" % (results.mean(), results.std()))
```

which is

```
1 mean = 0.764; std = 0.011
```

A good model should produce a high score (in this case, accuracy close to 1) and low standard deviation. A high standard deviation means the model is not very consistent with different test sets.

Putting everything together, the following is the complete code:

```
1 import copy
2
3 import numpy as np
4 import pandas as pd
5 import torch
6 import torch.nn as nn
7 import torch.optim as optim
8 from sklearn.preprocessing import LabelEncoder
9 from skorch import NeuralNetBinaryClassifier
10 from sklearn.model_selection import StratifiedKFold, train_test_split, cross_val_score
11
12 # Read data
13 data = pd.read_csv("sonar.csv", header=None)
14 X = data.iloc[:, 0:60]
15 y = data.iloc[:, 60]
16
17 # Binary encoding of labels
18 encoder = LabelEncoder()
19 encoder.fit(y)
20 y = encoder.transform(y)
21
22 # Convert to 2D PyTorch tensors
23 X = torch.tensor(X.values, dtype=torch.float32)
24 y = torch.tensor(y, dtype=torch.float32)
25
26 # Define the model
27 class SonarClassifier(nn.Module):
28     def __init__(self):
29         super().__init__()
30         self.layer1 = nn.Linear(60, 60)
31         self.act1 = nn.ReLU()
32         self.layer2 = nn.Linear(60, 60)
33         self.act2 = nn.ReLU()
34         self.layer3 = nn.Linear(60, 60)
35         self.act3 = nn.ReLU()
36         self.output = nn.Linear(60, 1)
37
38     def forward(self, x):
39         x = self.act1(self.layer1(x))
40         x = self.act2(self.layer2(x))
41         x = self.act3(self.layer3(x))
42         x = self.output(x)
43         return x
44
45 # create the skorch wrapper
46 model = NeuralNetBinaryClassifier(
47     SonarClassifier,
48     criterion=torch.nn.BCEWithLogitsLoss,
49     optimizer=torch.optim.Adam,
50     lr=0.0001,
51     max_epochs=150,
52     batch_size=10,
53     verbose=False
54 )
55
56 # k-fold
57 kfold = StratifiedKFold(n_splits=5, shuffle=True)
58 results = cross_val_score(model, X, y, cv=kfold)
59 print("mean = %.3f; std = %.3f" % (results.mean(), results.std()))
```

In comparison, the following is an equivalent implementation with a neural network model in scikit-learn:

```
1 from sklearn.model_selection import StratifiedKFold
2 from sklearn.model_selection import cross_val_score
3 from sklearn.neural_network import MLPClassifier
4 import numpy as np
5
6 # load dataset
7 data = pd.read_csv("sonar.csv", header=None)
8 # split into input (X) and output (Y) variables, in numpy arrays
```

```

9 X = data.iloc[:, 0:60].values
10 y = data.iloc[:, 60].values
11
12 # binary encoding of labels
13 encoder = LabelEncoder()
14 encoder.fit(y)
15 y = encoder.transform(y)
16
17 # create model
18 model = MLPClassifier(hidden_layer_sizes=(60,60,60), activation='relu',
19                       max_iter=150, batch_size=10, verbose=False)
20
21 # evaluate using 10-fold cross validation
22 kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=seed)
23 results = cross_val_score(model, X, y, cv=kfold)
24 print("mean = %.3f; std = %.3f" % (results.mean(), results.std()))

```

Which you should see how skorch is to make a drop-in replacement of scikit-learn model with a model from PyTorch.

Grid Search Deep Learning Model Parameters

The previous example showed how easy it is to wrap your deep learning model from PyTorch and use it in functions from the scikit-learn library. In this example, you will go a step further. The function that you specify to the model argument when creating the `NeuralNetBinaryClassifier` or `NeuralNetClassifier` wrapper can take many arguments. You can use these arguments to further customize the construction of the model. In addition, you know you can provide arguments to the `fit()` function.

In this example, you will use grid search to evaluate different configurations for your neural network model and report on the combination that provides the best estimated performance. To make it interesting, let's modify the PyTorch model such that it takes a parameter to decide how deep you want it to be:

```

1 class SonarClassifier(nn.Module):
2     def __init__(self, n_layers=3):
3         super().__init__()
4         self.layers = []
5         self.acts = []
6         for i in range(n_layers):
7             self.layers.append(nn.Linear(60, 60))
8             self.acts.append(nn.ReLU())
9             self.add_module(f"layer{i}", self.layers[-1])
10            self.add_module(f"act{i}", self.acts[-1])
11        self.output = nn.Linear(60, 1)
12
13    def forward(self, x):
14        for layer, act in zip(self.layers, self.acts):
15            x = act(layer(x))
16        x = self.output(x)
17        return x

```

In this design, we hold the hidden layers and their activation functions in Python lists. Because the PyTorch components are not immediate attributes of the class, you will not see them in `model.parameters()`. That will be a problem on training. This can be mitigated by using `self.add_module()` to register the components. An alternative is to use `nn.ModuleList()` instead of a Python list, so that you provided enough clues to tell where to find the components of the model.

The skorch wrapper is still the same. With it, you can have a model compatible to scikit-learn. As you can see, there are parameters to set up the deep learning model as well as training parameters such as learning rate (`lr`) specified in the wrapper, you have many possible variations. The `GridSearchCV` function from scikit-learn is to provide grid search cross validation. You can provide a list of values for each parameter and ask scikit-learn to try out **all combinations** and report the best set of parameters according to the metric you specified. An example is as follows:

```

1 from sklearn.model_selection import GridSearchCV
2
3 model = NeuralNetBinaryClassifier(
4     SonarClassifier,
5     criterion=torch.nn.BCEWithLogitsLoss,
6     optimizer=torch.optim.Adam,
7     lr=0.0001,
8     max_epochs=150,
9     batch_size=10,
10    verbose=False
11 )
12
13 param_grid = {
14     'module__n_layers': [1, 3, 5],
15     'lr': [0.1, 0.01, 0.001, 0.0001],
16     'max_epochs': [100, 150],
17 }
18
19 grid_search = GridSearchCV(model, param_grid, scoring='accuracy', verbose=1, cv=3)
20 result = grid_search.fit(X, y)

```

You passed in `model` to `GridSearchCV()`, which is a skorch wrapper. You also passed in `param_grid`, which specified to vary:

- the parameter `n_layers` in the PyTorch model (i.e., the `SonarClassifier` class), that controls the depth of the neural network
- the parameter `lr` in the wrapper, that controls the learning rate at the optimizer
- the parameter `max_epochs` in the wrapper, that controls the number of training epochs to run

Note the use of double underscore to pass on parameters to the PyTorch model. In fact, this allows you to configure other parameters too. For example, you can set up `optimizer__weight_decay` to pass on `weight_decay` parameters to the Adam optimizer (which is for setting up L2 regularization).

Running this can take a while to compute because it tries all combinations, each evaluated with 3-fold cross validation. You do not want to run this often but it can be useful for you to design models.

After the grid search is finished, the performance and combination of configurations for the best model are displayed, followed by the performance of all combinations of parameters, as below:

```
1 print("Best: %f using %s" % (result.best_score_, result.best_params_))
2 means = result.cv_results_['mean_test_score']
3 stds = result.cv_results_['std_test_score']
4 params = result.cv_results_['params']
5 for mean, stdev, param in zip(means, stds, params):
6     print("%f (%f) with: %r" % (mean, stdev, param))
```

It gives:

```
1 Best: 0.649551 using {'lr': 0.001, 'max_epochs': 150, 'module__n_layers': 1}
2 0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 100, 'module__n_layers': 1}
3 0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 100, 'module__n_layers': 3}
4 0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 100, 'module__n_layers': 5}
5 0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 150, 'module__n_layers': 1}
6 0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 150, 'module__n_layers': 3}
7 0.533678 (0.003611) with: {'lr': 0.1, 'max_epochs': 150, 'module__n_layers': 5}
8 0.644651 (0.062160) with: {'lr': 0.01, 'max_epochs': 100, 'module__n_layers': 1}
9 0.567495 (0.049728) with: {'lr': 0.01, 'max_epochs': 100, 'module__n_layers': 3}
10 0.533678 (0.003611) with: {'lr': 0.01, 'max_epochs': 100, 'module__n_layers': 5}
11 0.615804 (0.061966) with: {'lr': 0.01, 'max_epochs': 150, 'module__n_layers': 1}
12 0.620290 (0.078243) with: {'lr': 0.01, 'max_epochs': 150, 'module__n_layers': 3}
13 0.533678 (0.003611) with: {'lr': 0.01, 'max_epochs': 150, 'module__n_layers': 5}
14 0.635335 (0.108412) with: {'lr': 0.001, 'max_epochs': 100, 'module__n_layers': 1}
15 0.582126 (0.058072) with: {'lr': 0.001, 'max_epochs': 100, 'module__n_layers': 3}
16 0.563423 (0.136916) with: {'lr': 0.001, 'max_epochs': 100, 'module__n_layers': 5}
17 0.649551 (0.075676) with: {'lr': 0.001, 'max_epochs': 150, 'module__n_layers': 1}
18 0.558178 (0.071443) with: {'lr': 0.001, 'max_epochs': 150, 'module__n_layers': 3}
19 0.567909 (0.088623) with: {'lr': 0.001, 'max_epochs': 150, 'module__n_layers': 5}
20 0.557971 (0.041416) with: {'lr': 0.0001, 'max_epochs': 100, 'module__n_layers': 1}
21 0.587026 (0.079951) with: {'lr': 0.0001, 'max_epochs': 100, 'module__n_layers': 3}
22 0.606349 (0.092394) with: {'lr': 0.0001, 'max_epochs': 100, 'module__n_layers': 5}
23 0.563147 (0.099652) with: {'lr': 0.0001, 'max_epochs': 150, 'module__n_layers': 1}
24 0.534023 (0.057187) with: {'lr': 0.0001, 'max_epochs': 150, 'module__n_layers': 3}
25 0.634921 (0.057235) with: {'lr': 0.0001, 'max_epochs': 150, 'module__n_layers': 5}
```

This might take about 5 minutes to complete on your workstation executed on the CPU (rather than GPU). Running the example shows the results below. You can see that the grid search discovered that using a learning rate of 0.001 with 150 epochs and only a single hidden layer achieved the best cross-validation score of approximately 65% on this problem.

In fact, you can see if you can improve the result by first standardizing input features. Since the wrapper allows you to use PyTorch model with scikit-learn, you can also use the scikit-learn's standardizer in realtime, and create a machine learning pipeline:

```
1 from sklearn.pipeline import Pipeline, FunctionTransformer
2 from sklearn.preprocessing import StandardScaler
3
4 pipe = Pipeline([
5     ('scaler', StandardScaler()),
6     ('float32', FunctionTransformer(func=lambda X: torch.tensor(X, dtype=torch.float32),
7                                 validate=False)),
8     ('sonarmodel', model.initialize()),
9 ])
```

The new object pipe you created is another scikit-learn model that works just like the `model` object, except a standard scaler is applied before the data is passed on to the neural network. Therefore you can run a grid search on this pipeline, with a little tweak on the way parameters are specified:

```
1 param_grid = {
2     'sonarmodel__module__n_layers': [1, 3, 5],
3     'sonarmodel__lr': [0.1, 0.01, 0.001, 0.0001],
4     'sonarmodel__max_epochs': [100, 150],
5 }
6
7 grid_search = GridSearchCV(pipe, param_grid, scoring='accuracy', verbose=1, cv=3)
8 result = grid_search.fit(X, y)
9 print("Best: %f using %s" % (result.best_score_, result.best_params_))
10 means = result.cv_results_['mean_test_score']
11 stds = result.cv_results_['std_test_score']
12 params = result.cv_results_['params']
13 for mean, stdev, param in zip(means, stds, params):
14     print("%f (%f) with: %r" % (mean, stdev, param))
```

Two key points to note here: Since PyTorch models are running on 32-bit floats by default but NumPy arrays are usually 64-bit floats. These data types are not aligned, but scikit-learn's scaler always return you a NumPy array. Therefore you need to do type conversion in the middle of the pipeline, using a `FunctionTransformer` object.

Moreover, in a scikit-learn pipeline, each step is referred by a name, such as `scaler` and `sonarmodel`. Therefore, the parameters set for the pipeline need to carry the name as well. In the example above, we use `sonarmodel__module__n_layers` as a parameter for grid search. This refers to the `sonarmodel` part of the pipeline (which is your `skorch` wrapper), the `module` part therein (which is your PyTorch model), and its `n_layers` parameter. Note the use of double underscore for hierarchy separation.

Putting everything together, the following is the complete code:

```

1 import copy
2
3 import numpy as np
4 import pandas as pd
5 import torch
6 import torch.nn as nn
7 import torch.optim as optim
8 from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split, cross_val_score
9 from sklearn.pipeline import Pipeline, FunctionTransformer
10 from sklearn.preprocessing import StandardScaler, LabelEncoder
11 from skorch import NeuralNetBinaryClassifier
12
13 # Read data
14 data = pd.read_csv("sonar.csv", header=None)
15 X = data.iloc[:, 0:60]
16 y = data.iloc[:, 60]
17
18 # Binary encoding of labels
19 encoder = LabelEncoder()
20 encoder.fit(y)
21 y = encoder.transform(y)
22
23 # Convert to 2D PyTorch tensors
24 X = torch.tensor(X.values, dtype=torch.float32)
25 y = torch.tensor(y, dtype=torch.float32)
26
27
28 class SonarClassifier(nn.Module):
29     def __init__(self, n_layers=3):
30         super().__init__()
31         self.layers = []
32         self.acts = []
33         for i in range(n_layers):
34             self.layers.append(nn.Linear(60, 60))
35             self.acts.append(nn.ReLU())
36             self.add_module(f"layer{i}", self.layers[-1])
37             self.add_module(f"act{i}", self.acts[-1])
38         self.output = nn.Linear(60, 1)
39
40     def forward(self, x):
41         for layer, act in zip(self.layers, self.acts):
42             x = act(layer(x))
43         x = self.output(x)
44         return x
45
46 model = NeuralNetBinaryClassifier(
47     SonarClassifier,
48     criterion=torch.nn.BCEWithLogitsLoss,
49     optimizer=torch.optim.Adam,
50     lr=0.0001,
51     max_epochs=150,
52     batch_size=10,
53     verbose=False
54 )
55
56 pipe = Pipeline([
57     ('scaler', StandardScaler()),
58     ('float32', FunctionTransformer(func=lambda X: torch.tensor(X, dtype=torch.float32),
59                                validate=False)),
60     ('sonarmodel', model.initialize()),
61 ])
62
63 param_grid = {
64     'sonarmodel__module__n_layers': [1, 3, 5],
65     'sonarmodel__lr': [0.1, 0.01, 0.001, 0.0001],
66     'sonarmodel__max_epochs': [100, 150],
67 }
68
69 grid_search = GridSearchCV(pipe, param_grid, scoring='accuracy', verbose=1, cv=3)
70 result = grid_search.fit(X, y)
71 print("Best: %f using %s" % (result.best_score_, result.best_params_))
72 means = result.cv_results_['mean_test_score']
73 stds = result.cv_results_['std_test_score']
74 params = result.cv_results_['params']
75 for mean, stdev, param in zip(means, stds, params):
76     print("%f (%f) with: %r" % (mean, stdev, param))

```

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

Online Resources

- [skorch documentation](#)
- [Stratified K-Folds cross-validator](#). [scikit-learn documentation](#).
- [Grid search cross-validator](#). [scikit-learn documentation](#).
- [Pipeline](#). [scikit-learn documentation](#)

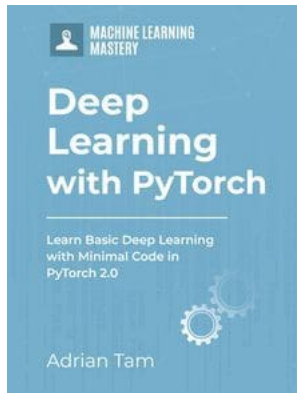
Summary

In this chapter, you discovered how to wrap your PyTorch deep learning models and use them in the scikit-learn general machine learning library. You learned:

- Specifically how to wrap PyTorch models so that they can be used with the scikit-learn machine learning library.
- How to use a wrapped PyTorch model as part of evaluating model performance in scikit-learn.
- How to perform hyperparameter tuning in scikit-learn using a wrapped PyTorch model.

You can see that using scikit-learn for standard machine learning operations such as model evaluation and model hyperparameter optimization can save a lot of time over implementing these schemes yourself. Wrapping your model allowed you to leverage powerful tools from scikit-learn to fit your deep learning models into your general machine learning process.

Get Started on Deep Learning with PyTorch!



Learn how to build deep learning models

...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
[Deep Learning with PyTorch](#)

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

[SEE WHAT'S INSIDE](#)

[Share](#)

[Share](#)

[Post](#)

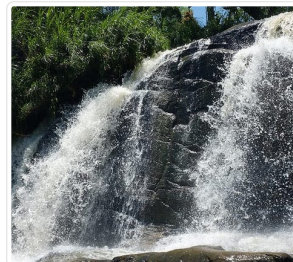
More On This Topic



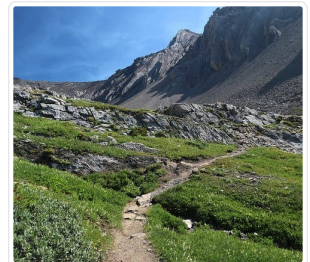
[Use Keras Deep Learning Models with Scikit-Learn in Python](#)



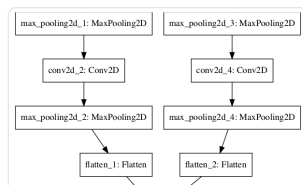
[Use Weight Regularization to Reduce Overfitting of...](#)



[How to Use Metrics for Deep Learning with Keras in Python](#)



[How to Use Word Embedding Layers for Deep Learning...](#)



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.

[View all posts by Adrian Tam →](#)

< [Building a Regression Model in PyTorch](#)

[How to Grid Search Hyperparameters for PyTorch Models](#) >

10 Responses to *Use PyTorch Deep Learning Models with scikit-learn*



james February 14, 2023 at 1:50 pm #

REPLY ↩

Thanks for the tutorial.

I faced the following error when executing the code.

TypeError Traceback (most recent call last)

Cell In[15], line 7

4 y = encoder.transform(y)

6 # Convert to 2D PyTorch tensors

—> 7 X = torch.tensor(X.values, dtype=torch.float32)

8 y = torch.tensor(y, dtype=torch.float32)

TypeError: can't convert np.ndarray of type numpy.object_. The only supported types are: float64, float32, float16, complex64, complex128, int64, int32, int16, int8, uint8, and bool.



Adrian Tam February 15, 2023 at 4:29 am #

REPLY ↩

Check what was X before this line. Probably you read some non-number into it.



Geoff Hardy March 26, 2023 at 2:04 am #

REPLY ↩

Hi James!

Thanks for the tutorial... just a typo comment... FunctionTransformer should be imported from sklearn.preprocessing ???



James Carmichael March 26, 2023 at 10:29 am #

REPLY ↩

Hi Geoff... You are correct! Thank you for the feedback!



zhao hongwei April 15, 2023 at 11:27 pm #

REPLY ↩

how to use the gridsearchcv and neuralnetclassifier optimize the weight_decay and lr_decay



James Carmichael April 16, 2023 at 9:26 am #

REPLY ↩

Hi Zhao... You can learn more here:

**WuGang** February 15, 2024 at 7:08 pm #

REPLY ↩

Why is the same hyperparameter trained by MLPClassifier so much better than by torch

**James Carmichael** February 16, 2024 at 10:32 am #

REPLY ↩

Hi WuGang...What are the differences in accuracy? Is this difference seen on average of multiple executions of training?

**shadow_** February 18, 2024 at 10:09 pm #

REPLY ↩

It seems that pipe does not need to call the initialization method of model, directly written ('sonarmodel', model) can be, because GridSearchCV has automatic initialization, plus initialize some cases will be prone to serialization problems

**James Carmichael** February 19, 2024 at 8:21 am #

REPLY ↩

Hi shadow_...Thank you for your feedback!

Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT

**Welcome!**I'm *Jason Brownlee* PhDand I **help developers** get results with **machine learning**.[Read more](#)**Never miss a tutorial:**

Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Building a Regression Model in PyTorch

Loving the Tutorials?

The **Deep Learning with PyTorch** EBook
is where you'll find the ***Really Good*** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. Visit our corporate [website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved